

Cours : Kubernetes sous contrainte de ressources

Owner	 louis reynouard
Tags	
Created time	May 1, 2026 4:40 PM

Pourquoi la contrainte de ressources est un sujet d'architecture

Dans un environnement cloud avec un budget, la réponse habituelle à un problème de performance est d'augmenter les ressources allouées. Cette approche masque les problèmes réels : un service mal dimensionné consomme ce qu'on lui donne, quelle que soit la quantité. Un pipeline inefficace sera simplement inefficace à plus grande échelle.

Travailler dans un environnement contraint force à répondre à des questions qu'on évite autrement : de combien de mémoire ce service a-t-il réellement besoin ? Que se passe-t-il quand deux services se disputent les mêmes ressources CPU ? Où est le goulot d'étranglement du système complet quand la charge augmente ?

Dans ce cours, Minikube sera lancé avec `--cpus=4 --memory=6144`. Ces valeurs définissent la capacité physique totale du nœud. C'est le plafond absolu du cluster.

Le ResourceQuota de votre namespace est volontairement inférieur pour deux raisons. D'abord, Kubernetes lui-même consomme des ressources sur le nœud (kubelet, API server, etcd, kube-proxy) : environ 0.5 à 1 CPU et 500 Mo à 1 Go selon l'activité.

Ensuite, des opérations comme un RollingUpdate ou un redémarrage après OOMKill nécessitent une marge temporaire au-delà des requests déclarées.

Allouer 100% d'un nœud à une application est une mauvaise pratique en production pour exactement ces raisons. La différence entre la capacité du nœud et le quota de votre namespace finance ces besoins système.

1. Le modèle de ressources Kubernetes

Requests et limits : deux rôles distincts

Kubernetes distingue deux valeurs pour chaque ressource dans un conteneur.

requests est la quantité de ressource que Kubernetes **réserve** pour ce conteneur sur le nœud. L'ordonnanceur utilise cette valeur pour décider sur quel nœud placer un pod. Si aucun nœud ne dispose des ressources demandées, le pod reste en état **Pending** indéfiniment. La **request** est une **garantie minimum**, pas un plafond.

limits est la quantité maximale que le conteneur peut consommer. Ce qui se passe en cas de dépassement dépend de la ressource.

Pour le CPU : Kubernetes applique du **throttling**. Le processus continue de tourner mais ses appels CPU sont ralenties. Le pod n'est pas tué, mais ses performances dégradent silencieusement. C'est le mode d'échec le plus difficile à détecter car rien ne plante.

Pour la mémoire : Kubernetes envoie un signal **OOMKill** au conteneur. Le pod est redémarré immédiatement. Si cela se produit régulièrement, le pod entre en état **CrashLoopBackOff** et Kubernetes applique un délai exponentiel entre les redémarrages.

```
resources:
  requests:
    cpu: "200m"      # 200 millicores = 0.2 CPU
    memory: "256Mi"  # 256 mébiotets
  limits:
    cpu: "500m"      # plafond à 0.5 CPU, throttling au-delà
    memory: "512Mi"  # plafond à 512 Mo, OOMKill au-delà
```

La notation `m` pour le CPU signifie millicores. $1000m = 1$ CPU logique. Un conteneur avec `requests.cpu: "200m"` se voit garantir 20% d'un CPU.

Raisonner sur le dimensionnement avant d'avoir des mesures

La question pratique : comment choisir des valeurs de `requests` quand on ne sait pas encore combien le service consomme ?

L'une des approches correctes est itérative. On commence par déployer sans `requests` ni `limits`, on envoie une charge représentative, et on observe avec `kubectl top pods`. Mais avant même cette étape, on peut raisonner par la nature du service:

- Un service qui charge un modèle de deep learning en mémoire (MobileNetV2, EfficientNet, DistilBERT) consomme la quasi-totalité de sa mémoire au démarrage lors du chargement des poids. La consommation au démarrage est souvent le pic de mémoire, pas la consommation sous charge. Un modèle qui prend 350 Mo de RAM au chargement n'en consommera pas beaucoup plus pendant l'inférence sur des images individuelles.
- Un service de preprocessing qui ne charge pas de modèle consomme surtout du CPU lors des transformations (redimensionnement, normalisation) et peu de mémoire sauf si les images sont très grandes.
- Un service de monitoring qui ne fait que compter et stocker des métriques en mémoire consomme peu des deux.

Ce raisonnement par type de service permet d'arriver à une première estimation raisonnable, qu'on affine ensuite avec les mesures réelles.

Interpréter kubectl top pods

```
kubectl top pods -n mon-namespace
```

La sortie ressemblera à :

NAME	CPU (cores)	MEMORY (bytes)
preprocessing-7d9f8b-xk2p4	45m	128Mi

inference-6c4b9d-p7qs1	312m	418Mi
monitoring-5a3c7e-r8nt2	8m	64Mi

CPU(cores) est la consommation instantanée en millicores. Une valeur de 45m signifie que le pod utilise 4.5% d'un CPU en ce moment. Si cette valeur approche régulièrement de votre `limits.cpu`, le throttling est probable.

MEMORY(bytes) est la consommation mémoire du processus. Si cette valeur dépasse régulièrement 80% de votre `limits.memory`, un OOMKill est probable lors d'un pic.

La règle de dimensionnement issue de la pratique : fixer `requests` à 70-80% du pic observé sous charge normale, et `limits` à 120-130% du pic observé. Cela laisse une marge pour les variations sans (trop) gaspiller des ressources.

2. ResourceQuota : le plafond du namespace

Un `ResourceQuota` fixe un plafond global pour un namespace. La somme des `requests` et des `limits` de tous les pods ne peut pas dépasser ces valeurs.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: projet-quota
  namespace: mon-namespace
spec:
  hard:
    requests.cpu: "3500m"
    requests.memory: "5Gi"
    limits.cpu: "3500m"
    limits.memory: "5Gi"
```

Effet concret : si la somme des `requests.cpu` de tous les pods dépasse 3500m, tout nouveau pod est rejeté avec l'erreur `exceeded quota`. Le scheduler refuse de le placer, même si des ressources physiques sont disponibles sur le nœud.

Attention : le quota est calculé sur les `requests` déclarées, pas sur la consommation réelle. Un pod qui déclare `requests.cpu: "1000m"` mais qui ne consomme réellement que 100m occupe quand même 1000m du quota.

```
kubectl describe resourcequota projet-quota -n mon-namespace
```

Name:	projet-quota	
Namespace:	mon-namespace	
Resource	Used	Hard
-----	----	----
limits.cpu	1200m	3500m
limits.memory	1792Mi	5Gi
requests.cpu	800m	3500m
requests.memory	1024Mi	5Gi

La colonne `Used` reflète la somme des valeurs déclarées dans les manifests des pods actifs, pas la consommation mesurée par `kubectl top`. La colonne `Hard` indique le plafond défini dans le `ResourceQuota`. C'est la valeur que vous ne pouvez pas dépasser. Si `Used` atteint `Hard`, tout nouveau pod est rejeté jusqu'à ce qu'un pod existant soit supprimé.

3. LimitRange : les bornes par conteneur

Le `ResourceQuota` contrôle le total du namespace. Le `LimitRange` contrôle les valeurs individuelles de chaque conteneur.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: projet-limits
  namespace: mon-namespace
spec:
  limits:
    - type: Container
```

```
default:
  cpu: "500m"
  memory: "512Mi"
defaultRequest:
  cpu: "100m"
  memory: "128Mi"
max:
  cpu: "2000m"
  memory: "2Gi"
min:
  cpu: "50m"
  memory: "64Mi"
```

default et **defaultRequest** : si un conteneur ne déclare pas de **limits** ou de **requests**, Kubernetes applique ces valeurs automatiquement. Sans **LimitRange**, un pod sans **limits** peut consommer toutes les ressources du nœud. **default** définit la valeur appliquée pour **limits** si absente. **defaultRequest** définit la valeur appliquée pour **requests** si absente. Si vous déclarez un **default** sans **defaultRequest**, Kubernetes fixe automatiquement **requests** = **limits**, ce qui réserve plus de ressources que nécessaire.

max et **min** : un conteneur ne peut pas déclarer une valeur hors de ces bornes. Kubernetes rejette le pod à la création si les valeurs sont hors plage.

4. L'impact du RollingUpdate sur les ressources : un calcul à faire avant de déployer

C'est le point que les documentations officielles n'expliquent pas clairement et qui cause des surprises sur un cluster contraint.

Lors d'un **RollingUpdate**, Kubernetes démarre les nouveaux pods **avant** de terminer les anciens. Pendant la transition, le namespace héberge temporairement deux versions du service en parallèle. Si votre pod d'inférence déclare **requests.memory: "1Gi"**, et que vous avez deux replicas, le RollingUpdate consomme

temporairement jusqu'à `2 × 1Gi = 2Gi` rien que pour ce service pendant la transition.

Le paramètre `maxSurge` contrôle combien de pods supplémentaires peuvent exister pendant la mise à jour.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # au plus 1 pod de plus que le nombre désiré
      maxUnavailable: 0   # aucun pod indisponible pendant la mise à jour
```

Avec `maxSurge: 1` et un Deployment à 1 replica, le RollingUpdate consomme temporairement `2 × requests` de ce pod. Si votre quota ne laisse pas cette marge, le nouveau pod reste en `Pending` et la mise à jour est bloquée indéfiniment.

`maxUnavailable` contrôle combien de pods peuvent être indisponibles pendant la mise à jour. Avec `maxUnavailable: 0`, Kubernetes ne tue aucun ancien pod avant que le nouveau soit Running. Pas de downtime, mais nécessite du quota supplémentaire (d'où l'importance de `maxSurge`).

Avec `maxUnavailable: 1`, Kubernetes peut tuer un ancien pod immédiatement. Bref downtime mais aucune ressource supplémentaire nécessaire, ce qui peut être préférable sur un cluster contraint.

Le calcul à faire avant de choisir sa stratégie :

quota disponible - ressources actuelles de tous les services = marge disponible pour le surge

Si cette marge est inférieure aux `requests` d'un pod, utiliser `Recreate` à la place, avec le downtime que cela implique.

```
spec:
  strategy:
```

```
type: Recreate # supprime tous les pods avant d'en créer de nouveaux
```

Ce choix et son calcul justificatif sont attendus dans l'ADR.

5. Namespace dédié et isolation

```
# Créer le namespace
kubectl create namespace projet-TRIGRAMME

# Appliquer les contraintes
kubectl apply -f k8s/quota.yaml -n projet-TRIGRAMME
kubectl apply -f k8s/limitrange.yaml -n projet-TRIGRAMME

# Définir le namespace par défaut pour la session courante
kubectl config set-context --current --namespace=projet-TRIGRAMME

# Vérifier l'état du quota
kubectl describe resourcequota -n projet-TRIGRAMME
kubectl describe limitrange -n projet-TRIGRAMME
```

6. Communication inter-services

Dans votre projet, le service de preprocessing, le service d'inférence et le service de monitoring tournent dans des pods distincts. Ils doivent s'appeler mutuellement. Le preprocessing envoie les données à l'inférence, le monitoring reçoit les métriques des deux. Ces appels ne passent pas par l'extérieur du cluster : ils transitent via le réseau interne Kubernetes.

Pour que cela fonctionne, chaque service expose un objet Service de type ClusterIP qui lui donne un nom DNS stable, indépendant de l'IP éphémère du pod. C'est ce mécanisme que vous utiliserez pour écrire les URLs entre vos services dans votre code.

Chaque `Service` Kubernetes de type `ClusterIP` est accessible depuis les autres pods via son nom DNS interne : `nom-du-service.namespace.svc.cluster.local`. Dans le même namespace, le nom court suffit.

```
apiVersion: v1
kind: Service
metadata:
  name: preprocessing-svc
spec:
  type: ClusterIP
  selector:
    app: preprocessing
  ports:
    - port: 8001
      targetPort: 8001
```

Le service d'inférence appelle le preprocessing via `http://preprocessing-svc:8001`. Le service de monitoring expose ses métriques sur son propre port via ClusterIP, et peut être consulté depuis l'extérieur avec `kubectl port-forward` pendant les tests.

Pour exposer le service d'inférence à l'extérieur du cluster afin de recevoir les requêtes du script de charge, utilisez `minikube service` :

```
minikube service inference-svc -n projet-TRIGRAMME --url
```

Cette commande retourne l'URL publique accessible depuis votre machine. C'est cette URL que vous passez au paramètre `--url` du script de charge.

Note importante : si vous utilisez `localhost` ou l'IP d'un pod directement dans votre code pour appeler un autre service, cela ne fonctionnera pas. Les pods ont des IPs éphémères qui changent à chaque redémarrage. Utilisez toujours le nom du Service Kubernetes.

7. Diagnostiquer les états d'échec fréquents sur cluster contraint

- **Pending** avec "Insufficient cpu" ou "Insufficient memory" : le scheduler ne trouve pas assez de ressources disponibles. Soit le quota est dépassé (vérifier `kubectl describe resourcequota`), soit les ressources physiques du nœud sont saturées (vérifier `kubectl top nodes`).
- **OOMKilled** : le conteneur a dépassé sa `limits.memory`. Identifiable via `kubectl describe pod NOM` dans la section `Last State : Reason: OOMKilled`. Corriger en augmentant `limits.memory` ou en réduisant la consommation du service (lazy loading du modèle, batch size réduit). Dans le cadre du tp attention de rester dans les limites définies !
- **CrashLoopBackOff** : redémarrages répétés. Peut indiquer des OOMKills répétés ou une erreur applicative. Vérifier avec `kubectl logs NOM --previous` pour voir les logs du run précédent.
- **RollingUpdate bloqué** : un nouveau pod reste en **Pending** pendant une mise à jour. Indique que le quota ne laisse pas la marge nécessaire pour le surge. Vérifier l'état du quota pendant la mise à jour et adapter la stratégie.

```
# Commandes de diagnostic essentielles
kubectl get pods -n mon-namespace # état g
énéral
kubectl top pods -n mon-namespace # consomm
ation en direct
kubectl describe pod NOM -n mon-namespace # détail
+ events
kubectl logs NOM -n mon-namespace # logs co
urants
kubectl logs NOM --previous -n mon-namespace # logs ru
n précédent
kubectl describe resourcequota -n mon-namespace # état du
quota
kubectl rollout status deployment NOM -n mon-namespace # sui
vi d'un déploiement
```

8. Questions de raisonnement

Certaines des questions suivantes vous seront posées à l'oral:

- Votre service d'inférence déclare `requests.memory: "800Mi"` et `limits.memory: "1Gi"`. Vous observez avec `kubecttl top` une consommation de 950Mi sous charge. Que va-t-il se passer lors d'un pic de 20% au-dessus de la charge normale ? Quelle action préventive pouvez-vous prendre sans modifier les modèles ?
- Votre quota total est de 5 Go. Vous avez trois services avec des `requests.memory` de 1Gi, 2Gi et 512Mi. Vous voulez effectuer un RollingUpdate du service à 2Gi avec `maxSurge: 1`. Est-ce que le quota le permet ? Montrez le calcul.
- Votre service de preprocessing est throttlé en CPU : `kubecttl top` montre 480m de consommation pour une `limits.cpu` de 500m. La latence du service d'inférence en aval augmente proportionnellement. Sans toucher au quota total, comment redistribuez-vous les ressources entre les services pour corriger ce problème ?